

Investigating Side-Channel Attack (SCA) Resistance and Threshold Implementation (TI) in Post-Quantum Hash-Based Signature: SLH-DSA.

By Adiyat Abubakirov, Tina Carter, and
Justin Williams

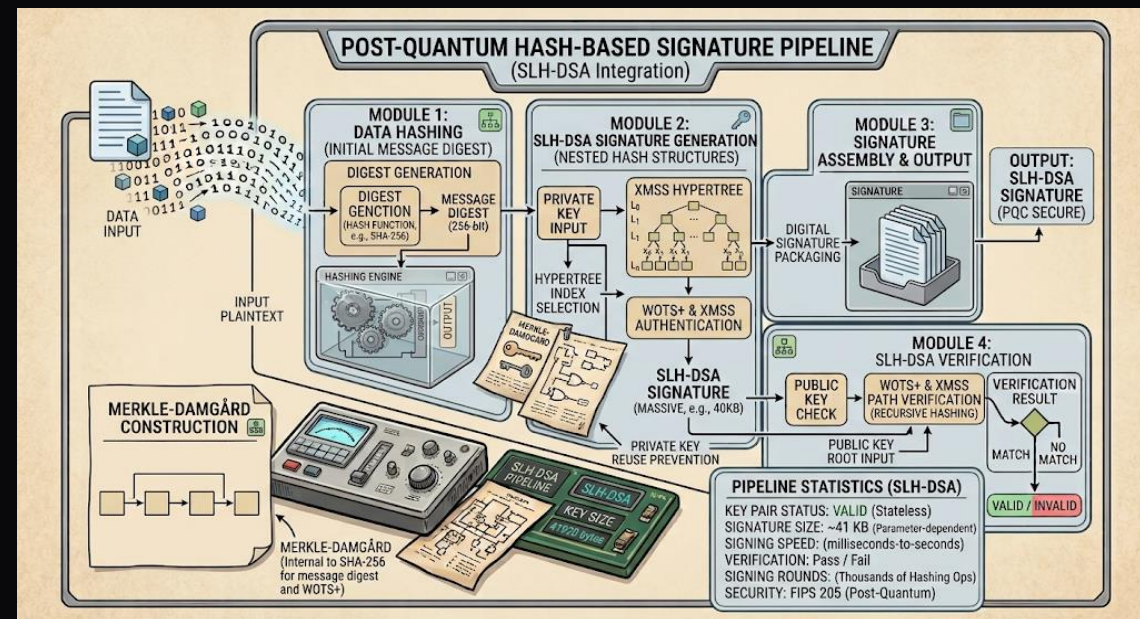
SLotH

```

10 [ RAM / Memory ] <=====(System Bus)===== [ CPU (RISC-V) ] <=====(System Bus)===== [ Generic Hash Core ]
9 | ^ | ^ |
8 | | | |
7 (1) CPU reads SK.seed word-by-word (LEAKS KEY!) | | (4) CPU reads 32-byte Hash Output |
6 | | | |
5 (2) CPU manually formats ADRS, PK.seed, & Padding | | (5) CPU increments loop counter |
4 (Wastes cycles on byte-shifting & alignment) | | (6) CPU writes output back to RAM |
3 | | | |
2 | | | |
1 +-----+ +-----+
11 [Repeat this loop 10,000+ times for ONE signature!]

23 [ CPU (RISC-V) ]
22 |
21 (1) Write Trigger & Iteration Count to Hardware Register
20 | (e.g., KECC_CHNS = 15 for Winternitz Chain)
19 |
18 |
17 |=====|
16 | SLoTH Hardware Accelerator (Internal High-Speed Silicon Fabric) |
15 | | |
14 [ Dedicated Hardware Registers ] [ Autonomous State Machines ]
13 - S256_SEED (Holds PK.seed) ----> - Auto-Padding & Formatting Engine
12 - S256_ADRS (Holds Domain Addr) ----> - Winternitz Chain Loop Controller
11 - KECC_SKSD (Holds SK.seed) ----> - Single-Cycle Key Injection Vault
10 | |
9 +-----+
8 | (Zero-Copy Internal Routing) |
7 | |
6 | [ Hash Core (Keccak/SHA2) ] |
5 | (100% Utilization, No CPU) |
4 |=====|
3 |
2 | (2) CPU reads Final Result
1 |
24 [ RAM / Memory ]

```



Intellectual Merit

Problem:

- New standard for quantum-resistant digital signatures (SLH-DSA - Stateless Hash-Based Digital Signature Standard) is slow on standard computers.
- Requires millions of repetitive, heavy hash calculations (Winternitz chains) & complex message formatting.

Why it matters:

- Quantum computers are approaching and standard chips' security is unable to protect from quantum computers.

Contributions:

- Authors build SLoTH, a custom hardware accelerator that completely automates the algorithm's repetitive "grunt work" (like message padding and chaining).
- **100x to 300x speedup** compared to standard methods.
- ISSUE: Standard software secretly leaks its master key via power consumption.
- Successfully engineered a hardware shield (Threshold Implementation) to block the leak.

Impact

The paper sets up a great move in the right direction to maintain the **integrity** and **security** of firmware as we enter an age of quantum computing. **Secure boot** is a critical process to laptops, phones, and computers of all kinds, and firmware attacks on them could be devastating. The proposed ideas encourage further **collaboration between hardware and software** design when creating cryptographic capabilities. Additionally, the Hardware Accelerator built/talked about in the paper (SLoTH), is **open source**, so anyone can collaborate, build, and improve on its functions.

Our Progress

Background Research:

RoT for SoC; existing computer implementations of signature verification (with/without hash).

Coding Project Outline:

1. Core PRF Implementation

- SLH-DSA PRF (using SHAKE256) and the Winternitz chain iteration logic in Python.
- Verify against NIST Known Answer Tests (KATs) to ensure our cryptographic math is 100% compliant with FIPS 205

2. Leakage Modelling Engine

- The simulation engine that models power consumption
- **Hamming Weight** of the intermediate variables (which directly ties to the Abstract Algebra and GF(2) XOR operations we studied in **Lectures 3 & 4**).
- Model both the "leaky" CPU mode and the "protected" SLoTH mode.

3. TVLA Statistical Framework

- Use Welch's t-test to compare fixed and random key trace sets and plot to show t-value spikes as 'critical value' leaks

We Are Here